

LLM Agents for Cross-Platform Prediction Market Arbitrage

Abhi Wadhwa
University of Southern California
abhiw@usc.edu

Summer 2026

Abstract

Prediction markets on Kalshi and Polymarket price the same real-world events under different naming conventions, fee structures, and settlement rules. Cross-platform arbitrage requires solving three chained problems: matching semantically equivalent contracts across platforms, detecting fee-adjusted pricing discrepancies from order book data, and executing trades under non-atomic settlement risk. I develop a formal treatment of cross-platform prediction market microstructure—fee models, arbitrage definitions, and the contract matching problem—and build a hybrid NLP pipeline (regex extraction, Sentence-BERT embeddings, rapidfuzz string matching, optional LLM verification) for cross-platform contract resolution. I evaluate the matching pipeline on a synthetic benchmark of 5,000 market pairs and report results. Real-data evaluation on live Kalshi and Polymarket markets, LLM agent experiments with actual API calls, and paper trading simulations are in progress and will be reported in a subsequent version.

1 Introduction

I got interested in prediction markets when I noticed the same NFL game priced differently on two platforms. The Kansas City Chiefs–Buffalo Bills matchup on October 12, 2025: Kalshi had the Chiefs at 47 cents, Polymarket at 52 cents. Five cents of spread on a binary contract that pays a dollar. After fees, that’s still roughly three cents of profit per contract, risk-free in theory. I bought YES on Kalshi and NO on Polymarket, pocketed \$2.80 on a hundred contracts, and started wondering how often this happens.

It happens a lot. Prediction markets have grown from academic curiosities and small-stakes novelties into billion-dollar platforms. Kalshi, the first CFTC-regulated designated contract market for event contracts, now lists thousands of markets spanning sports, politics, weather, and economic indicators. Polymarket, built on the Polygon blockchain, processed \$22.88 billion in trading volume in 2025 alone. With that kind of volume comes disagreement—the same event priced by different populations of traders, under different fee structures, with different settlement mechanisms. Disagreement means arbitrage.

[Saguillo et al. \(2025\)](#) documented approximately \$40 million in realized cross-platform arbitrage profits between April 2024 and April 2025, with the bulk concentrated during high-attention events like U.S. presidential debates and NFL playoffs. That number only counts opportunities that someone actually captured. The total addressable spread, including opportunities missed due to latency, capital constraints, or simply not looking, is presumably larger.

But capturing arbitrage across prediction markets isn’t like capturing it across equity exchanges. There are three distinct skill types required, and they have almost nothing in common.

The first is **NLP matching**. Kalshi encodes events as structured tickers like `KXNFLGAME-25OCT12-KCBUF`. Polymarket uses free-text descriptions like “Will the Kansas City Chiefs beat the Buffalo Bills on October 12, 2025?” Matching these requires understanding sports abbreviations, date formats, and the implicit mapping between team codes and full names. For politics and economics, the matching problem gets harder—Kalshi might list “GDP Q3 2025 above 2.5%” while Polymarket phrases it as “Will US GDP growth exceed 2.5% in 2025 Q3?” Same event, completely different surface forms.

The second skill is **fee-adjusted pricing**. Both platforms charge taker fees, but the formulas differ. Kalshi uses a nonlinear formula with a ceiling function. Polymarket uses a continuous multiplier. An opportunity that looks profitable before fees can vanish after them. Getting the arithmetic right is non-negotiable.

The third is **execution under uncertainty**. Prediction market arbitrage is non-atomic. You fill one leg on Kalshi, then the other leg on Polymarket. Between those two fills—maybe a few seconds, maybe a few minutes—the price on the second platform can move. The gap is where real money gets lost.

These three skills map neatly onto three different capabilities. Matching is a language task. Fee calculation is a math task. Execution is a decision-making task under sequential uncertainty. This decomposition makes prediction market arbitrage a natural testbed for evaluating LLM agents: each subtask is independently measurable, the outcome is denominated in dollars, and there’s a natural rule-based baseline for comparison.

The question I’m trying to answer isn’t whether LLMs can do arbitrage. It’s more granular than that. Where in the pipeline do they add value, and where do they fail? My hypothesis is that LLMs will excel at matching (it’s fundamentally a language understanding task) and struggle at fee calculation (it’s fundamentally an arithmetic task). Testing this hypothesis on real data is in progress.

I make four contributions:

1. A formal treatment of cross-platform prediction market microstructure: fee models for both Kalshi and Polymarket, three types of arbitrage (cross-platform, implied same-platform, monotonicity violations), and the contract matching problem as entity resolution.
2. A hybrid NLP pipeline for cross-platform contract matching, evaluated on a synthetic benchmark with precision/recall analysis across market categories.
3. A synthetic benchmark of 5,000 cross-platform market pairs for development and unit testing of matching and arbitrage detection systems.
4. A framework for LLM agent evaluation—task decomposition, prompt design, rule-based baselines, and evaluation metrics—designed but not yet tested with real LLM API calls on live market data.

The rest of the paper is organized as follows. Section 2 covers prediction market microstructure, fee models, and a formal treatment of arbitrage types. Section 3 describes the system architecture, including the hybrid matching pipeline and order book model. Section 4 presents the LLM agent design and rule-based baselines. Section 5 describes the synthetic benchmark and reports matching results on synthetic data. Section 6 reports the current status of each component—what’s verified, what’s preliminary, and what’s planned. Section 7 discusses implications and future directions.

2 Background and Market Microstructure

2.1 Binary Contracts

Definition 2.1 (Binary Contract). A binary contract on event E is a financial instrument that pays \$1 if E occurs and \$0 otherwise. A YES position at price $p \in (0, 1)$ profits $1 - p$ if E occurs and loses p if E doesn't. A NO position at price q profits $1 - q$ if E doesn't occur and loses q if it does.

Both Kalshi and Polymarket trade binary contracts, but the infrastructure is different in every other respect.

Kalshi is a CFTC-regulated designated contract market (DCM). Tickers follow a structured convention: `KXNFLGAME-25OCT12-KCBUF` encodes the category (NFL game), date (October 12, 2025), and teams (Kansas City vs. Buffalo). The platform provides REST and WebSocket APIs authenticated via RSA-PSS signatures. Order books report YES-side bids and asks; NO-side prices are implicit.

Polymarket settles on the Polygon blockchain. Events are identified by condition IDs generated through the UMA oracle system. The platform exposes three API layers: the Gamma API for market metadata, the CLOB API for order book data, and the Data API for historical trades. Authentication uses EIP-712 typed signatures. Order books report both YES and NO tokens as independent tradeable assets.

A critical structural difference: Kalshi returns bid-only data for binary contracts. If the YES side has a best bid of p cents, the implied NO ask is $100 - p$ cents. Polymarket reports YES and NO as separate tokens with independent order books, and there's no guarantee their prices sum to exactly \$1 at every moment—which itself creates same-platform arbitrage opportunities.

2.2 Fee Models

Fees eat arbitrage profits. Getting them wrong by even a fraction of a cent per contract can flip a profitable trade into a losing one. The two platforms use materially different fee formulas.

Definition 2.2 (Kalshi Taker Fee). For a trade of C contracts at price P (in dollars, $P \in (0, 1)$), the Kalshi taker fee in cents is:

$$f_K(C, P) = \lceil 0.07 \cdot C \cdot P(1 - P) \rceil \text{ cents.} \quad (1)$$

The ceiling function $\lceil \cdot \rceil$ means every trade pays at least 1 cent in fees, regardless of size.

Definition 2.3 (Polymarket Taker Fee). For a trade of Q tokens at price P , the Polymarket taker fee in USDC is:

$$f_P(Q, P) = \gamma \cdot Q \cdot P(1 - P), \quad (2)$$

where γ is a category-dependent fee rate, approximately 0.02 for sports markets.

Both fees share the $P(1 - P)$ kernel, but Kalshi's ceiling function introduces a discontinuity that matters for small trades.

Proposition 2.4. *Both fee functions are maximized at $P = 0.5$ and vanish as $P \rightarrow 0$ or $P \rightarrow 1$.*

Proof. The function $g(P) = P(1 - P)$ has derivative $g'(P) = 1 - 2P$, which equals zero at $P = 1/2$. Since $g''(P) = -2 < 0$, the point $P = 1/2$ is a maximum. At the boundaries, $g(0) = g(1) = 0$. Both f_K and f_P are non-negative scalar multiples of $g(P)$ (up to the ceiling in f_K), so they inherit this behavior. $\square$

Remark 2.5. This means fees are highest precisely where arbitrage is most common—at mid-probability events where cross-platform disagreement tends to concentrate. Events near $P = 0.5$ attract diverse opinions and heavy trading, producing the largest price discrepancies across platforms. But those same events incur the largest fees. It’s a natural moat protecting market makers from arbitrageurs.

To put concrete numbers on this: buying 100 contracts at $P = 0.50$ on Kalshi costs $\lceil 0.07 \times 100 \times 0.25 \rceil = \lceil 1.75 \rceil = 2$ cents. On Polymarket with $\gamma = 0.02$, the same trade costs $0.02 \times 100 \times 0.25 = 0.50$ USDC. Kalshi’s fee is four times larger at the midpoint. At $P = 0.10$, Kalshi charges $\lceil 0.07 \times 100 \times 0.09 \rceil = \lceil 0.63 \rceil = 1$ cent, while Polymarket charges $0.02 \times 100 \times 0.09 = 0.18$ USDC. The ratio narrows at extreme prices, but Kalshi is always weakly more expensive due to the ceiling.

2.3 Arbitrage Types

I consider three types of arbitrage opportunities. They differ in structure, risk profile, and detection difficulty.

Definition 2.6 (Direct Cross-Platform Arbitrage). For the same event E , suppose Platform A offers YES at ask price a and Platform B offers NO at ask price b . The gross edge per contract is:

$$e_{\text{gross}} = 1 - a - b. \quad (3)$$

The net edge after fees for n contracts is:

$$e_{\text{net}}(n) = 1 - a - b - \frac{f_A(n, a)}{n} - \frac{f_B(n, b)}{n}. \quad (4)$$

An arbitrage opportunity exists when $e_{\text{net}}(n) > 0$ for some $n \geq 1$.

Definition 2.7 (Implied Same-Platform Arbitrage). On a single platform, if the best YES ask p_{yes} and best NO ask p_{no} satisfy

$$p_{\text{yes}} + p_{\text{no}} < 1 - \frac{f(n, p_{\text{yes}}) + f(n, p_{\text{no}})}{n} \quad (5)$$

for some n , then buying both YES and NO guarantees a profit.

Same-platform arbitrage is rarer on Kalshi (a centralized exchange enforces $p_{\text{yes}} + p_{\text{no}} \geq 1$ mechanically) but common on Polymarket, where YES and NO tokens trade independently. The two sides can temporarily decouple, especially during high-volatility news events.

Definition 2.8 (Monotonicity Violation). For a set of threshold markets on the same underlying variable X with thresholds $T_1 < T_2 < \dots < T_k$, the prices must satisfy:

$$\mathbb{P}(X > T_1) \geq \mathbb{P}(X > T_2) \geq \dots \geq \mathbb{P}(X > T_k). \quad (6)$$

A violation $\mathbb{P}(X > T_i) < \mathbb{P}(X > T_{i+1})$ for some i signals mispricing.

Monotonicity violations show up in economic threshold markets. Kalshi might list five markets on Q3 GDP growth: above 1%, above 1.5%, above 2%, above 2.5%, above 3%. If “above 2.5%” trades at 45 cents while “above 2%” trades at 42 cents, that’s a violation—the less restrictive threshold should be priced weakly higher. These violations are usually small (1–3 cents) and short-lived.

Remark 2.9 (Non-Atomic Settlement Risk). Unlike futures arbitrage where both legs can fill simultaneously through a single exchange, prediction market legs execute sequentially. I buy YES on Kalshi, then turn around and buy NO on Polymarket. Between those two fills—call it a gap of δ seconds—the Polymarket price can move. If the NO price rises by more than the original edge during the gap, the “arbitrage” becomes a loss. This execution risk is the main reason real arbitrage profits are smaller than theoretical edge suggests.

2.4 The Matching Problem

Before any arbitrage detection can happen, I need to know which Kalshi market corresponds to which Polymarket market. This is an entity resolution problem.

Formally: let $\mathcal{K} = \{k_1, k_2, \dots, k_m\}$ be the set of active Kalshi tickers and $\mathcal{P} = \{p_1, p_2, \dots, p_n\}$ the set of active Polymarket events. I want to find all pairs $(k_i, p_j) \in \mathcal{K} \times \mathcal{P}$ that refer to the same real-world event.

The mapping isn't one-to-one. A single Polymarket event ("Will the Chiefs beat the Bills?") might correspond to multiple Kalshi tickers if Kalshi lists both the game outcome and the point spread. Conversely, a Kalshi ticker for "Will inflation exceed 3% in 2025?" might have no Polymarket equivalent if Polymarket hasn't listed that specific threshold.

Several factors make the matching hard:

- **Naming conventions.** Kalshi tickers use structured abbreviations (KC for Kansas City, BUF for Buffalo). Polymarket uses full names, sometimes with articles and prepositions ("the Kansas City Chiefs").
- **Temporal encoding.** Kalshi encodes dates as 25OCT12 (YYMMDD). Polymarket uses natural language ("October 12, 2025" or "Oct 12" or "10/12/25").
- **Granularity.** Kalshi might list 10 threshold levels for GDP; Polymarket might list 5 different ones. The overlapping thresholds need to be identified.
- **Ambiguity.** Political events are the worst. "Will Biden comment on the debate?" versus "Will the President respond to the presidential debate controversy?"—these might or might not refer to the same event depending on context and timing.

Example 2.10. The ticker KXNFLGAME-25OCT12-KCBUF decomposes as:

- KX: Kalshi exchange prefix.
- NFLGAME: category (NFL game outcome).
- 25OCT12: date (October 12, 2025).
- KC: Kansas City Chiefs.
- BUF: Buffalo Bills.

The corresponding Polymarket event is: "Will the Kansas City Chiefs beat the Buffalo Bills on October 12, 2025?" Matching these requires a lookup table from team abbreviations to full names, date parsing, and the understanding that "NFLGAME" implies a binary win/loss outcome.

The matching problem is the gateway to everything else. Miss a match and you miss an arbitrage opportunity. False-match and you trade two unrelated events against each other—which isn't arbitrage, it's just gambling.

3 System Architecture

3.1 Hybrid Matching Pipeline

The matching system processes candidates through four stages, each designed to handle a different slice of the difficulty distribution. Easy matches get caught early; hard ones propagate to more expensive layers.

Stage 1: Rule-Based Extraction. Regex patterns parse Kalshi tickers into structured fields: category, teams/entities, date, and threshold (if applicable). Polymarket descriptions are processed with keyword extraction: named entity recognition identifies teams, dates, and numerical thresholds. Both sides produce a canonical tuple (c, e_1, e_2, d, t) where c is category, e_1, e_2 are entities, d is date, and t is threshold. Exact field matches at this stage produce high-confidence pairs immediately. On the synthetic benchmark, about 60% of true matches are caught here—mostly sports, where the naming is structured on both sides.

Stage 2: Sentence-BERT Embedding. For unmatched candidates, I normalize both the Kalshi ticker (expanding abbreviations using a lookup table) and the Polymarket description (lowercasing, removing articles), then encode both with `all-MiniLM-L6-v2` (Reimers and Gurevych, 2019). Pairs with cosine similarity above 0.85 are classified as matches. This catches cases where the field extraction fails—unusual team abbreviations, non-standard date formats, or events that don’t decompose neatly into the (c, e_1, e_2, d, t) schema.

Stage 3: Fuzzy String Fallback. Residual candidates (similarity between 0.60 and 0.85) are processed with `rapidfuzz` (Bachmann, 2021) token-set-ratio. This handles typos, minor wording differences, and cases where the embedding model assigns moderate similarity to genuinely matching pairs. The threshold here is 80% token-set-ratio.

Stage 4: LLM Verification. Candidates that pass Stage 3 but fall in the uncertain zone (token-set-ratio between 80% and 95%, or Sentence-BERT similarity between 0.70 and 0.90) get sent to an LLM. The prompt includes both descriptions and asks for a binary match/no-match judgment with a one-sentence explanation. This is the most expensive stage—roughly $50\times$ the latency of Stage 2—so it only processes the ambiguous tail.

Each stage rejects clear non-matches before the next fires. The cascade structure means the LLM only sees the hard cases. On the synthetic benchmark, about 8% of candidate pairs reach Stage 4.

3.2 Order Book Model

I model each side of each market as an L2 order book: sorted arrays of (price, volume) tuples for bids and asks.

$$\text{Book}_{\text{ask}} = \{(p_1, v_1), (p_2, v_2), \dots, (p_L, v_L)\} \quad \text{with } p_1 < p_2 < \dots < p_L, \quad (7)$$

where L is the number of price levels. Each level represents a standing limit order. To fill n contracts, you walk up the book: consume v_1 contracts at p_1 , then v_2 at p_2 , and so on until the order is filled or the book is exhausted.

The effective fill price for n contracts is the volume-weighted average:

$$\bar{p}(n) = \frac{\sum_{i=1}^k p_i \cdot \min(v_i, r_i)}{\sum_{i=1}^k \min(v_i, r_i)}, \quad (8)$$

where $r_1 = n$ and $r_{i+1} = r_i - \min(v_i, r_i)$, with k being the first level where $r_k \leq v_k$.

Cross-platform synchronization is a real problem. Kalshi’s WebSocket updates arrive every 100–500ms. Polymarket’s CLOB updates can lag by 1–3 seconds during high-volume periods. Books older than 30 seconds are unreliable for arbitrage decisions—prices can move 2–3 cents in that window.

3.3 Fee-Adjusted Edge Calculation

The net edge for a cross-platform trade of n contracts, buying YES on Platform A at effective price $\bar{a}(n)$ and NO on Platform B at effective price $\bar{b}(n)$, is:

$$e_{\text{net}}(n) = 1 - \bar{a}(n) - \bar{b}(n) - \frac{f_A(n, \bar{a}(n))}{n} - \frac{f_B(n, \bar{b}(n))}{n}. \quad (9)$$

This is a concave function of n for two reasons. First, $\bar{a}(n)$ and $\bar{b}(n)$ are weakly increasing in n (walking up the book means worse prices). Second, Kalshi’s ceiling function makes the per-contract fee $f_K(n, P)/n$ weakly decreasing in n —the ceiling’s 1-cent overhead is amortized over more contracts. So there’s a tension: larger size improves per-unit fee efficiency but worsens fill prices.

The optimal position size n^* satisfies:

$$n^* = \arg \max_{n \in \{1, \dots, N_{\max}\}} n \cdot e_{\text{net}}(n), \quad (10)$$

where $N_{\max}$ is the minimum depth available across both books. In practice, I don’t optimize over all possible sizes. I compute $e_{\text{net}}(n)$ for $n \in \{1, 10, 25, 50, 100\}$ and pick the best. The optimization surface is smooth enough that this coarse grid works.

I set a minimum edge threshold of 2%. If $e_{\text{net}}(n^*) < 0.02$, the opportunity is skipped. This threshold covers execution risk (the non-atomic settlement gap from Remark 2.9), slippage from stale books, and the possibility that the matcher produced a false positive.

4 LLM Agent Design

4.1 Task Decomposition

I decompose the arbitrage pipeline into three formal tasks, each testable in isolation.

Definition 4.1 (Task T_1 : Match Verification). Given two market descriptions (d_K, d_P) —one from Kalshi, one from Polymarket—output $\hat{y} \in \{0, 1\}$ indicating whether they refer to the same real-world event.

Definition 4.2 (Task T_2 : Trade Decision). Given an arbitrage opportunity $o = (\text{match pair, order books, edge, fees})$ and current portfolio state $s = (\text{positions, capital, daily PnL})$, output a decision $\hat{d} \in \{\text{trade, skip}\}$ and, if trading, a position size $\hat{n}$.

Definition 4.3 (Task T_3 : Risk Assessment). Given portfolio state s and a set of pending opportunities $\{o_1, \dots, o_k\}$, output risk metrics: estimated portfolio variance, maximum loss scenario, and a recommendation to continue trading or pause.

T_1 is the language task. T_2 is the quantitative decision task. T_3 is a monitoring task that gates the other two. In a fully autonomous agent, all three run continuously; the evaluation framework I’ve built tests them separately and then together.

4.2 Agent Architecture

For each task, the LLM receives a structured prompt containing all information needed for the decision. I don’t include the full prompts here—they’re available in the code repository—but I describe the key design choices.

For T_1 , the prompt presents the Kalshi ticker (expanded into natural language using the abbreviation lookup table) alongside the Polymarket description. The LLM is asked: “Do these two descriptions refer to the same event? Answer YES or NO. Then explain your reasoning in one sentence.” The explanation is logged but not used algorithmically.

For T_2 , the prompt includes: the matched pair descriptions, both order books (top 5 levels), the computed gross and net edge, the fee breakdown, current positions, available capital, and the day’s running PnL. The LLM outputs a JSON object: `{‘action’: ‘trade’/‘skip’, ‘size’: int, ‘reason’: string}`. The reason field provides interpretability.

Here’s the key test embedded in T_2 : can the LLM correctly verify the fee computation? The prompt provides raw inputs and asks the agent to confirm or reject the precomputed fee. For example:

Kalshi fee for 100 contracts at $P = 0.47$: computed as $\lceil 0.07 \times 100 \times 0.47 \times 0.53 \rceil = \lceil 1.7402 \rceil = 2$ cents. Is this correct?

The answer is yes. I expect this to be a failure point for LLMs—multi-step floating-point multiplication combined with a discrete ceiling function is exactly the kind of computation where approximate reasoning breaks down. But I don’t yet have real error rates. The experiments described in Section 6.3 will measure this.

For T_3 , the prompt presents the full portfolio: all open positions, their entry prices, current market prices, and total exposure. The LLM estimates worst-case loss (all positions go to zero on the losing side) and recommends whether to continue trading or stop for the day. This task is likely the easiest for the LLM—it’s mostly qualitative reasoning about concentration risk.

4.3 Rule-Based Baseline

The rule-based system serves as the comparison point. No LLM involvement at all.

Matching. Stages 1–3 of the hybrid pipeline (Section 3.1) with tuned thresholds: SBERT similarity ≥ 0.85 , rapidfuzz token-set-ratio $\geq 80\%$. No LLM verification (Stage 4 is skipped).

Trading. If $e_{\text{net}}(n^*) > 0.02$ and the market has sufficient depth (≥ 50 contracts at the best price on both sides), execute the trade. Position sizing uses fractional Kelly:

$$n = \min \left(\left\lfloor 0.25 \cdot \frac{e_{\text{net}}}{\sigma^2} \right\rfloor, N_{\text{max}} \right), \quad (11)$$

where σ^2 is the estimated variance of the outcome (for a binary at price p , $\sigma^2 = p(1 - p)$), and N_{max} is the position limit. The 0.25 multiplier is a quarter-Kelly scaling, standard practice for reducing variance at the cost of lower expected growth. Position limits are \$100 per market and \$10,000 total exposure.

Risk. Hard-coded stop-loss: if daily PnL drops below -5% of capital, stop trading for the day. Maximum correlation allowed between positions: no more than 30% of capital in a single category (sports, politics, etc.). These rules are simple, don’t require any NLP, and execute in microseconds.

4.4 Evaluation Metrics

Matching is evaluated with standard information retrieval metrics: precision (P), recall (R), and F1 score ($F1 = 2PR/(P + R)$). I compute these globally and per category.

Trading is evaluated with standard portfolio metrics:

- Cumulative PnL: total dollars earned net of all fees.

- Sharpe ratio: $S = \bar{r}/\sigma_r$, where $\bar{r}$ is mean daily return and σ_r is daily return standard deviation. Annualized by multiplying by $\sqrt{252}$.
- Maximum drawdown: largest peak-to-trough decline in cumulative PnL.
- Trade count and error count.

Decision quality measures the fraction of decisions that agree with the optimal ex-post decision:

$$\text{DQ} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\text{sign}(\hat{d}_i) = \text{sign}(d_i^*)], \quad (12)$$

where $\hat{d}_i$ is the agent’s decision (trade or skip) and d_i^* is the decision that would have been optimal given realized outcomes. This metric is noisy—some decisions are right but unlucky—so I report it alongside PnL, which integrates over both decision quality and sizing accuracy.

Status note. The evaluation framework described in this section is fully implemented in code: prompts are written, the rule-based baseline runs, and the metrics compute correctly on synthetic data. What’s missing is testing with real LLM API calls (GPT-4, Claude, etc.) on live market data. Section 6.3 describes the planned experiments.

5 Synthetic Benchmark

I construct a synthetic benchmark for development and unit testing. This section describes the data generation process and reports matching results on synthetic data. These results test the pipeline’s internal logic—whether the stages work correctly and compose as intended. They don’t tell us how the pipeline performs on real Kalshi and Polymarket markets, where naming conventions are messier, categories are more diverse, and the matching difficulty distribution is different. Real-data evaluation is described in Section 6.2 and planned in Section 6.3.

5.1 Synthetic Data Generation

I generate a synthetic benchmark rather than starting with live market data for two reasons. First, reproducibility: live markets change second by second, making experiments non-repeatable. Second, control: I can vary the difficulty of matching pairs, the size of arbitrage edges, and the liquidity of order books independently.

The benchmark contains 5,000 market pairs. The category distribution: 50% sports (2,500 pairs), 20% cryptocurrency (1,000), 15% politics (750), 15% economics (750). This roughly mirrors the volume distribution on Kalshi and Polymarket.

For each true match, I generate:

- A Kalshi-style ticker following the structured convention (`KXCATEGORY-YYMMDD-ENTITIES`).
- A Polymarket-style description in natural language.
- Two order books with 10 price levels each, spreads drawn uniformly from 1–5 cents, and depth at each level drawn from a log-normal distribution with mean 50 contracts.

Negative pairs (non-matches) are generated by corrupting one field of a true match: swapping teams (“Chiefs vs Bills” → “Chiefs vs Ravens”), shifting the date by one day, changing the threshold

Table 1: Cross-platform matching results on the **synthetic** benchmark (5,000 market pairs). Stage 4 uses a mock LLM verifier, not real LLM API calls. These numbers test pipeline logic, not real-world matching performance.

Method	Precision	Recall	F1
Regex only	0.891	0.723	0.798
Regex + SBERT	0.931	0.912	0.921
Regex + SBERT + Fuzzy	0.931	0.947	0.939
Full pipeline (+mock LLM)	0.978	0.943	0.960
Mock LLM only	0.854	0.912	0.882

(“above 2.5%” → “above 3.0%”), or replacing the category entirely. This produces hard negatives—pairs that share most fields but differ on a critical one. About 40% of the 5,000 pairs are true matches; 60% are negatives, of which half are hard negatives and half are easy negatives (completely different events).

5.2 Matching Results on Synthetic Data

I evaluate five matching configurations on the synthetic benchmark:

1. **Regex only**: Stage 1 alone.
2. **Regex + SBERT**: Stages 1–2.
3. **Regex + SBERT + Fuzzy**: Stages 1–3 (the full pipeline without LLM).
4. **Full pipeline**: All four stages (using a mock LLM verifier that applies string-similarity heuristics to simulate Stage 4 decisions).
5. **Mock LLM only**: Skip all rule-based stages; the mock verifier processes every candidate pair.

Important caveat: Stage 4 in this evaluation uses a mock LLM verifier, not real API calls to GPT-4 or Claude. The mock applies a string-similarity heuristic designed to approximate LLM behavior on easy cases. The numbers below therefore test the pipeline architecture and the first three stages, but don’t tell us how a real LLM would perform on Stage 4 verification. Real LLM evaluation is planned (Section 6.3).

The first three rows are the real result here. The hybrid pipeline without any LLM (Regex + SBERT + Fuzzy) hits 93.9% F1 on synthetic data, with balanced precision and recall. Each stage contributes incrementally: regex alone gets 79.8% F1 by catching the easy structured matches. Adding SBERT jumps to 92.1%—a 12.3-point gain—by handling cases where field extraction fails. Fuzzy matching adds another 1.8 points by catching the remaining near-misses.

The “Full pipeline” and “Mock LLM only” rows show what happens with the mock verifier. The mock boosts precision from 93.1% to 97.8% when used as Stage 4 of the cascade, but performs poorly (88.2% F1) when used alone—it doesn’t have the structured extraction that the first three stages provide. These numbers are useful for validating the cascade architecture but shouldn’t be interpreted as predictions of real LLM performance.

On the synthetic data, category-level performance follows expected patterns: sports matching is easiest (structured abbreviations map cleanly to full names), politics is hardest (paraphrasing

and ambiguous descriptions), with crypto and economics in between. But synthetic data likely understates the difficulty of real political event matching, where descriptions are more varied and context-dependent.

5.3 Ablation: Pipeline Stage Contribution

To quantify each pipeline stage’s contribution on the synthetic data, I measure:

- F1 improvement from adding each stage.
- Latency cost of each stage.
- The number of candidate pairs each stage processes (cascading filter effect).

The cascade design filters aggressively. Stage 1 resolves about 60% of true matches. Stage 2 catches another 30%. Stage 3 handles about 8%. Only the remaining 2% of ambiguous pairs would reach Stage 4 in the full pipeline. The latency profile is correspondingly skewed: regex extraction runs in microseconds, SBERT encoding takes 2–5ms per pair, fuzzy matching is comparable, and LLM verification (when using a real model) would average roughly 800ms per pair. The cascade structure makes the expensive Stage 4 practical by restricting its input to a small tail of hard cases.

6 Results and Status

This section gives a direct accounting of what’s done, what’s preliminary, and what’s planned. I don’t want to bury the status in hedged language. A reader should be able to scan this section and know exactly where the project stands.

6.1 Verified Components

The following components are implemented, tested, and correct:

Fee models. The Kalshi fee formula (Definition 2.2) and Polymarket fee formula (Definition 2.3) are implemented and verified against actual platform API documentation and fee schedules. The ceiling-function behavior in the Kalshi formula produces correct outputs for all tested inputs, including edge cases at extreme prices (P near 0 or 1) and small trade sizes ($C = 1$) where the ceiling dominates.

Arbitrage definitions. The three arbitrage types (cross-platform, implied same-platform, monotonicity violations) are correctly formalized and implemented as detection functions. Given correct price inputs, they identify opportunities accurately. The net edge calculation (Eq. 9) accounts for both fee models and order book depth.

Matching pipeline (Stages 1–3). The rule-based extraction, SBERT embedding, and fuzzy string matching stages all run correctly. On the synthetic benchmark, the three-stage pipeline without LLM achieves 93.9% F1 (Table 1). All seven Python files in the codebase run without errors.

Synthetic benchmark. The 5,000-pair dataset generates correctly with the specified category distribution and difficulty profile. It serves its purpose: development, debugging, and unit testing of the matching and detection logic.

Mathematical framework. The fee model analysis (Proposition 2.4), the fee moat observation (Remark 2.5), the order book model, and the position sizing formulation are all correct mathematical descriptions of real systems. These don’t depend on empirical validation—they’re derived from the platform specifications.

6.2 Preliminary Results: Live Market Data

I built a data fetcher that pulls real market listings from the Kalshi API (`api.elections.kalshi.com/trade-api/v2`) and the Polymarket Gamma API (`gamma-api.polymarket.com`), normalizes them, and runs the three-stage matching pipeline. Both endpoints are public and require no authentication for read access. The fetcher ran on May 25, 2026. Here’s what I found.

Market inventory. Kalshi returned 813 active markets. Polymarket returned 46 active markets (filtered to sports and crypto categories with active order books). The inventory asymmetry is itself a finding: Polymarket lists far fewer markets at any given time than Kalshi, which limits cross-platform matching opportunities.

Kalshi’s structural shift. Of 813 Kalshi markets, 782 (96.2%) were multi-leg parlay contracts under the `KXMVESPORTSMULTIGAMEEXTENDED` ticker family. These encode compound bets—“OKC wins by over 3.5 points AND over 219.5 points scored”—rather than simple binary outcomes. Only 31 markets (3.8%) were non-parlay. This is a structural change from the market design I assumed: the system architecture targets simple binary matching (one Kalshi event $\leftrightarrow$ one Polymarket event), but Kalshi’s current inventory is dominated by multi-leg contracts that have no single-event Polymarket equivalent.

Matching results. The pipeline produced 14 candidate matches, all in sports. The highest match confidence was 0.564 (an OKC Thunder parlay matched to “Will the Oklahoma City Thunder win the NBA Western Conference Finals?”). All 14 candidates were structurally incorrect matches: they paired multi-leg Kalshi parlays with single-outcome Polymarket markets. The pipeline correctly assigned low confidence scores—none exceeded 0.57—but didn’t reject them outright because the team name overlap (“Oklahoma City” appears in both) creates genuine semantic similarity.

Order book analysis. Of the 14 candidate matches, most had empty or illiquid Kalshi order books (best bid = 0, best ask = 1.00). One candidate showed a Kalshi ask of \$0.062 against a Polymarket bid of \$0.62, producing a nominal gross edge of 55.8 cents. But this is a false arbitrage: the Kalshi contract is a multi-leg parlay (OKC wins by 2.5+ AND total over 219.5 AND total over 230.5) while the Polymarket contract is a simple series-winner bet. They’re not the same event. The price discrepancy reflects the lower probability of the compound outcome, not a market inefficiency.

Key finding. The live data reveals a structural mismatch between current Kalshi and Polymarket inventories that my synthetic benchmark didn’t capture. Cross-platform arbitrage requires matching pairs of simple binary contracts on the same event, but Kalshi’s shift toward multi-leg parlays means most of its volume is in contracts that have no Polymarket equivalent. The arbitrage opportunity set may be smaller than the \$40 million figure from [Sagullo et al. \(2025\)](#) suggests—that estimate covered a period when both platforms listed more single-outcome events.

This doesn’t invalidate the pipeline or the LLM evaluation framework, but it changes the deployment picture. A production system would need a parlay decomposition stage: parse multi-leg Kalshi contracts into individual components, then match each component separately. I haven’t built that yet.

6.3 Planned Experiments

Five experiments remain. I list them in rough priority order:

1. **Hand-labeled matching benchmark on live markets ($N \geq 200$ pairs).** Fetch active markets from both platforms, hand-label matches and non-matches across categories (sports, politics, crypto, economics), and evaluate all five pipeline configurations on real data. This is the most important missing piece—synthetic matching performance doesn’t transfer reliably to real-world naming patterns.

2. **LLM agent evaluation with real API calls.** Run GPT-4 and Claude on all three tasks (T_1 : match verification, T_2 : trade decision, T_3 : risk assessment) using real market data as inputs. Measure the actual fee calculation error rate—I expect LLMs to fail on the Kalshi ceiling function, but I don’t yet have a number.
3. **Paper trading simulation with live order book data.** Run both the rule-based agent and LLM agent on a multi-week simulation using real order book snapshots from both platforms. This requires solving the cross-platform synchronization problem (matching timestamps between Kalshi WebSocket data and Polymarket CLOB snapshots).
4. **Fee calculation error rate measurement on real LLM outputs.** Present real fee computation problems (with actual prices and quantities from live markets) to multiple LLMs and measure: arithmetic error rate, ceiling omission rate, unit confusion rate. This is a focused sub-experiment of item 2 that can be run independently.
5. **Full backtest with historical market data.** If historical order book data is available (Kalshi provides some through their data API; Polymarket historical data requires scraping), run a retrospective simulation over 3–6 months to estimate realized PnL, Sharpe ratio, and drawdown for both agent types.

Until these experiments run, the claims I can make are limited: the pipeline architecture is sound, the fee math is correct, and the matching logic works on synthetic data. Whether the system finds real arbitrage, and whether LLMs help or hurt at each stage, is an open question.

7 Discussion

7.1 What the Synthetic Results Tell Us

The synthetic benchmark validates the pipeline architecture. Each stage adds value: regex handles the structured easy cases, SBERT catches semantic equivalences that regex misses, and fuzzy matching picks up the near-misses. The cascade filters aggressively enough that the expensive LLM stage (once real) will only see a small tail of ambiguous pairs. That’s the right design—you don’t want to spend 800ms per pair on candidates that regex could resolve in microseconds.

The 93.9% F1 on synthetic data for the three-stage pipeline (no LLM) is encouraging but not conclusive. Synthetic data is generated by rules, and the pipeline is also rule-based. There’s some circularity: I designed the negative pairs to be “hard” by corrupting single fields, which is exactly the kind of difficulty that field-matching excels at catching. Real negative pairs might be hard in different ways—different categories that happen to share entities, or different events on the same date between the same teams (e.g., a regular season game vs. a playoff game).

The matching pipeline’s real test comes when it runs on live data. I expect performance to drop, particularly on politics and economics markets where real Polymarket descriptions are more varied than my synthetic templates. Sports matching should hold up better because the naming is genuinely structured on both platforms.

7.2 Expected LLM Behavior

I have a clear hypothesis but no data yet. Let me state it plainly so the real experiments can confirm or reject it.

Matching (T_1): I expect LLMs to add value here, especially on politics and economics markets where paraphrasing and implicit references are common. The LLM should catch cases like “Will

the former president attend the debate?” matching “Trump debate attendance”—mappings that require world knowledge the rule-based pipeline doesn’t have. The gain will be largest on the hard tail of ambiguous pairs.

Fee calculation (T_2): I expect LLMs to fail here. The Kalshi fee formula requires multi-step floating-point multiplication followed by a ceiling function. Based on the existing literature on LLM arithmetic (Dziri et al., 2023; Qian et al., 2022), this is the kind of compositional computation where errors compound. The ceiling function adds a discrete cliff to an otherwise smooth calculation. I predict error rates in the 5–15% range for Kalshi fee computation and below 3% for Polymarket’s simpler formula. But these are predictions, not measurements.

State tracking (T_3): I expect LLMs to handle qualitative risk assessment passably—identifying concentration risk, flagging over-exposure to a single category. But I expect them to fail at precise capital tracking across sequential decisions, because each prompt call is stateless and the portfolio state is passed as text. The rule-based agent trivially tracks capital with a running counter.

The headline prediction: LLMs will help at the NLP layer and hurt at the quantitative layer. The net contribution to PnL depends on the ratio of language to math in the pipeline—and this pipeline has more math than language.

7.3 Design Implications

Even without real-data results, the system design points toward a hybrid architecture. LLMs for NLP—matching, information extraction from news feeds, interpreting settlement rule ambiguities. Deterministic code for everything quantitative—fee computation, edge calculation, position sizing, risk limits, order book parsing.

This matches the tool-use paradigm from Schick et al. (2023): LLMs should reason and plan; tools should compute. In this context, the “tools” are the fee calculator, the order book parser, and the Kelly sizing formula. The LLM’s job is to decide whether two events match and to interpret qualitative information (news, settlement rules, regulatory changes). Everything that can be specified as a formula should be.

A concrete system design:

1. **Ingestion layer:** rule-based. Parse API responses, build order books, compute derived quantities.
2. **Matching layer:** hybrid. Regex + SBERT + fuzzy for the easy 92% of pairs, LLM for the ambiguous 8%.
3. **Detection layer:** rule-based. Fee computation, edge calculation, opportunity scoring.
4. **Execution layer:** rule-based. Kelly sizing, position limits, stop-losses.
5. **Monitoring layer:** LLM. Summarize daily performance, flag unusual patterns, interpret news that might affect open positions.

The LLM appears at stages 2 and 5. Stages 1, 3, and 4 are pure code. This architecture is more complex than “just use an LLM for everything,” but it should be more reliable and cheaper (fewer API calls). Whether it actually is—that’s what the experiments will show.

7.4 Limitations

The biggest limitation is that this is, at present, a design paper with synthetic validation. The theory is sound, the code runs, and the pipeline works on fake data. But I don’t yet have the numbers that matter: real matching accuracy, real LLM error rates, or real PnL.

Synthetic data understates difficulty in specific ways. My benchmark generates order books from parametric distributions (log-normal depth, uniform spreads). Real order books have fat tails, clustering, and autocorrelation. Real markets have adversarial dynamics: market makers adapt to arbitrageurs, widen spreads when they detect systematic flow, and use information from one platform’s book to update quotes on the other. My synthetic data doesn’t capture any of this.

I’ve tested the pipeline against one mock LLM and zero real LLMs. Different models (GPT-4, Claude, Gemini) have different training distributions and might handle the ceiling function differently. A multi-model evaluation is planned but not yet done.

There’s no execution risk modeling with real data. In practice, you’d face rejected orders, partial fills, platform outages, and API rate limits. The Polymarket CLOB has minimum order sizes and tick sizes that I haven’t tested against.

Regulatory uncertainty looms over the whole enterprise. Kalshi is CFTC-regulated. Polymarket’s primary interface operates from outside the United States, and its regulatory status is ambiguous. Cross-platform arbitrage between a regulated and an unregulated venue raises legal questions for U.S.-based operators that I don’t address. The technical pipeline I describe works regardless of regulatory status, but deploying it is a different question.

Acknowledgments

Thanks to the prediction markets community on Twitter for making fee structures, API documentation, and arbitrage strategies public knowledge. The open discussion of market microstructure on Kalshi and Polymarket made this project possible.

References

- Max Bachmann. Rapidfuzz: Rapid fuzzy string matching. <https://github.com/maxbachmann/RapidFuzz>, 2021.
- Nouha Dziri, Ximing Lu, Melanie Sclar, Xiang Lorraine Li, Liwei Jiang, Bill Yuchen Lin, Sean Welleck, Peter West, Chandra Bhagavatula, Ronan Le Bras, Jena D Hwang, and Yejin Choi. Faith and fate: Limits of transformers on compositionality. In *Advances in Neural Information Processing Systems*, volume 36, 2023.
- Jing Qian, Hong Wang, Zekun Li, Shiyang Li, and Xifeng Yan. Limitations of language models in certain reasoning tasks. *arXiv preprint arXiv:2210.06726*, 2022.
- Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pages 3982–3992, 2019.
- Pedro Saguillo, Seyed Hossein Ghafouri, Lucianna Kiffer, and Guillermo Suarez-Tangil. Arbitrage in prediction markets. *arXiv preprint arXiv:2508.03474*, 2025.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36, 2023.